DATABASE CREATION AND TABLE IMPLEMENTATION

ASSIGNMENT 5

DATABASE MANAGEMENT SYSTEM

ALLIANCE UNIVERSITY

Department of Computer Applications

SUMMITED BY: BHUVANA S

REGISTER NO: 2511021240036

COURSE: BCA

YEAR: 2ND YEAR

Introduction

Database creation and table implementation are important steps in developing a database management system. SQL Data Definition Language (DDL) commands are used to create databases, define tables, specify data types, and establish relationships between tables. In this assignment, a Food Delivery Management System is implemented using MySQL. The database contains seven tables to manage customers, restaurants, menu items, orders, order items, payments, and delivery partners.

1. Creating the Database

- CREATE DATABASE Food_Delivery_DB;
- USE Food_Delivery_DB;

Explanation: CREATE DATABASE creates a new database, and USE selects it for creating and managing tables.

2. Creating the Required Tables

Table 1: CUSTOMER

```
CREATE TABLE CUSTOMER (
   Customer_ID INT PRIMARY KEY,
   Customer_Name VARCHAR(100),
   Phone VARCHAR(15),
   Email VARCHAR(100),
   Address VARCHAR(200));
```

Explanation: Customer_ID uniquely identifies each customer. The other columns store the customer's name, phone number, email, and address.

Table 2: RESTAURANT

```
CREATE TABLE RESTAURANT (
   Restaurant_ID INT PRIMARY KEY,
   Restaurant_Name VARCHAR(100),
   Phone VARCHAR(15),
   Rating DECIMAL(2,1),
```

Address VARCHAR(200));

Explanation: Restaurant_ID uniquely identifies each restaurant. The table also stores its name, contact number, rating, and address.

Table 3: MENU_ITEM

```
CREATE TABLE MENU_ITEM (
   Item_ID INT PRIMARY KEY,
   Restaurant_ID INT,
   Item_Name VARCHAR(100),
   Category VARCHAR(50),
   Price DECIMAL(10,2),
   Availability BOOLEAN,
   FOREIGN KEY (Restaurant_ID)
      REFERENCES RESTAURANT(Restaurant_ID));
```

Explanation: Item_ID is the primary key. Restaurant_ID is a foreign key that connects each menu item to its restaurant. One restaurant can have multiple menu items.

Table 4: DELIVERY_PARTNER

The DELIVERY_PARTNER table stores delivery partner details.

```
CREATE TABLE DELIVERY_PARTNER (
   Partner_ID INT PRIMARY KEY,
   Partner_Name VARCHAR(100),
   Phone VARCHAR(15));
```

Explanation: Partner_ID uniquely identifies each delivery partner. The table stores the partner's name and phone number. Separating partner details avoids repeating the same contact information for multiple orders.

Table 5: ORDERS

```
CREATE TABLE ORDERS (
   Order_ID INT PRIMARY KEY,
   Customer_ID INT,
   Restaurant_ID INT,
```

```
    Partner_ID INT,
    Order_Date DATE,
    Total_Amount DECIMAL(10,2),
    Order_Status VARCHAR(30),
    Delivery_Status VARCHAR(30),
    FOREIGN KEY (Customer_ID)
        REFERENCES CUSTOMER(Customer_ID),
    FOREIGN KEY (Restaurant_ID)
        REFERENCES RESTAURANT(Restaurant_ID),
    FOREIGN KEY (Partner_ID)
        REFERENCES DELIVERY_PARTNER(Partner_ID));
```

Explanation: Order_ID uniquely identifies each order. Customer_ID, Restaurant_ID, and Partner_ID are foreign keys that connect an order to the relevant customer, restaurant, and delivery partner. The table also stores the order date, total amount, order status, and delivery status.

Table 6: ORDER_ITEM

```
CREATE TABLE ORDER_ITEM (
    Order_Item_ID INT PRIMARY KEY,
    Order_ID INT,
    Item_ID INT,
    Quantity INT,
    Subtotal DECIMAL(10,2),
    FOREIGN KEY (Order_ID)
        REFERENCES ORDERS(Order_ID),
    FOREIGN KEY (Item_ID)
        REFERENCES MENU_ITEM(Item_ID));
```

Explanation: Order_Item_ID uniquely identifies each order item record. Order_ID connects it to an order, while Item_ID connects it to a menu item. Quantity and subtotal record the number of items ordered and their cost.

Table 7: PAYMENT

```
CREATE TABLE PAYMENT (
   Payment_ID INT PRIMARY KEY,
   Order_ID INT,
   Payment_Date DATE,
   Payment_Method VARCHAR(30),
   Amount DECIMAL(10,2),
   Payment_Status VARCHAR(30),
   FOREIGN KEY (Order_ID)
      REFERENCES ORDERS(Order_ID));
```

Explanation: Payment_ID uniquely identifies each payment record. Order_ID connects the payment to the corresponding order. The remaining columns store the payment date, method, amount, and status.

3. Defining Relationships Between Tables

The following relationships are established using primary keys and foreign keys.

Relationship	Type	Purpose
CUSTOMER → ORDERS	One-to-Many	One customer can place multiple orders.
RESTAURANT → MENU_ITEM	One-to-Many	One restaurant can offer multiple menu items.
RESTAURANT → ORDERS	One-to-Many	One restaurant can receive multiple orders.
DELIVERY_PARTNER → ORDERS	One-to-Many	One delivery partner can handle multiple orders over time.
ORDERS → ORDER_ITEM	One-to-Many	One order can contain multiple items
MENU_ITEM → ORDER_ITEM	One-to-Many	One menu item can appear in multiple order records.
ORDERS → PAYMENT	One-to-One	Each order is intended to have one payment record.

4. Verifying Table Creation

After creating the tables, execute the following SQL command to display the tables in the selected database.

SHOW TABLES;

Expected output:

Tables_in_Food_Delivery_DB
CUSTOMER
DELIVERY_PARTNER
MENU_ITEM
ORDERS
ORDER_ITEM
PAYMENT
RESTAURANT

To examine the structure of an individual table, execute:

DESCRIBE CUSTOMER;

The command displays the column names, data types, key information, and other column properties of the CUSTOMER table. The same command can be used with the other six tables.

Conclusion

The Food Delivery Management System database was designed using SQL DDL commands in MySQL. Seven tables were created to manage customers, restaurants, menu items, orders, order items, payments, and delivery partners. Primary keys and foreign keys were defined to identify records and establish relationships between tables. This database structure provides a foundation for inserting sample data, implementing integrity constraints, and executing SQL queries in the subsequent assignments.